

# Neural World Models for Quantum State Tomography

From Classical Shadow Tomography to Active Inference:  
Theory, Intuition, and Implementation

quantum-gym

May 16, 2026

## Contents

# 1 Overview

The `benchmark_world_models.py` script benchmarks three neural *world models* against a random baseline in an **active learning loop** for quantum state tomography. The central question is:

*Given a fixed budget of quantum measurements (shots), which model recovers the stabilizer structure of a cluster state fastest?*

The script sits at the intersection of three ideas:

1. **Classical Shadow Tomography** — efficient state estimation from random Pauli measurements.
2. **Active Inference** — choosing *which* measurements to perform next, guided by Expected Free Energy (EFE).
3. **Neural World Models** — parameterised models (NQS, VAE, MADE-VAE) that learn a compressed representation of the quantum state from measurement data.

## 2 Background: Classical Shadow Tomography

### 2.1 The Shadow Channel

Given an  $n$ -qubit state  $\rho$ , perform a random unitary  $U$  drawn from a fixed ensemble  $\mathcal{E}$  (e.g. tensor products of single-qubit Clifford unitaries), measure in the computational basis, and record outcome  $\hat{b} \in \{0, 1\}^n$ . The *classical shadow* is:

$$\hat{\sigma} = \mathcal{M}^{-1}\left(U^\dagger |\hat{b}\rangle\langle \hat{b}| U\right), \quad (1)$$

where  $\mathcal{M}$  is the *shadow channel*:

$$\mathcal{M}(\rho) = \mathbb{E}_{U \sim \mathcal{E}} \left[ U^\dagger \left( \sum_b \langle b| U \rho U^\dagger |b\rangle |b\rangle\langle b| \right) U \right]. \quad (2)$$

For the random single-qubit Pauli ensemble (each qubit independently measured in  $X$ ,  $Y$ , or  $Z$  with equal probability  $\frac{1}{3}$ ), the channel inverts as:

$$\mathcal{M}^{-1}(\cdot) = \bigotimes_{i=1}^n (3(\cdot) - \mathbf{I}). \quad (3)$$

### 2.2 Pauli Estimators

The unbiased estimator of a Pauli observable  $P = \bigotimes_{i=1}^n P_i$ ,  $P_i \in \{I, X, Y, Z\}$ , from a single shadow  $(b, s)$  is:

$$\hat{v}_P(b, s) = \begin{cases} \prod_{i: P_i \neq I} 3 s_i & \text{if } P_i = b_i \text{ for all } i \text{ where } P_i \neq I, \\ 0 & \text{otherwise.} \end{cases} \quad (4)$$

### 2.3 Stabilizer Fidelity

$$F_{\text{stab}}(\rho_{\text{model}}) = \frac{1}{n} \sum_{i=1}^n \frac{1 + \langle K_i \rangle_{\text{model}}}{2} \in [0, 1]. \quad (5)$$

### 3 What is an Autoregressive Model?

#### Analogy: Writing a sentence word by word

Imagine writing the sentence “*The cat sat on the mat*” one word at a time. When you write “*sat*”, you already know “*The cat*” came before. An autoregressive model does exactly this: it predicts each token (or bit) *conditioned on everything it has already generated*. It never looks ahead — only backward.

#### 3.1 The Chain Rule of Probability

Any joint probability distribution over  $n$  variables can be *exactly* factored using the chain rule:

$$p(s_1, s_2, \dots, s_n) = p(s_1) \cdot p(s_2 | s_1) \cdot p(s_3 | s_1, s_2) \cdots p(s_n | s_1, \dots, s_{n-1}). \quad (6)$$

This is not an approximation — it is always true. The question is *how* to parameterise each conditional. An autoregressive model uses a neural network for each conditional.

#### 3.2 A Simple 3-Qubit Example

Suppose we measure 3 qubits in basis  $b = (X, Z, Y)$  and get outcome  $s = (+1, -1, +1)$ .

An autoregressive model computes:

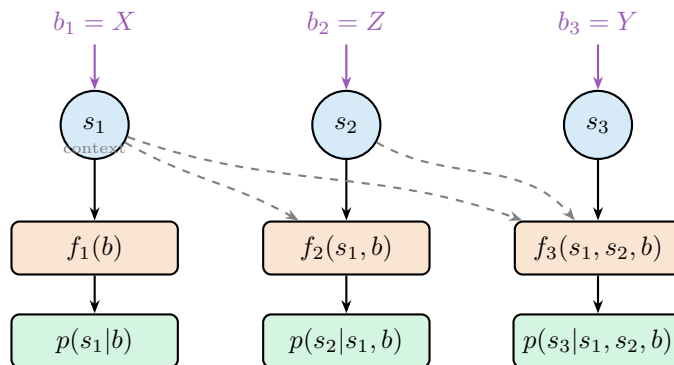
$$p(s_1 = +1 | b) = \sigma(f_1(b)), \quad (7)$$

$$p(s_2 = -1 | s_1 = +1, b) = \sigma(f_2(s_1, b)), \quad (8)$$

$$p(s_3 = +1 | s_1 = +1, s_2 = -1, b) = \sigma(f_3(s_1, s_2, b)), \quad (9)$$

where  $\sigma$  is the sigmoid function and  $f_i$  are small neural networks. The joint probability is then:

$$p(s | b) = p(s_1 | b) \cdot p(s_2 | s_1, b) \cdot p(s_3 | s_1, s_2, b). \quad (10)$$



#### Key Idea: The autoregressive property

Each network  $f_i$  only receives *past* outcomes  $s_1, \dots, s_{i-1}$  as context. This is the **autoregressive property**: no information from the future leaks into the prediction. It guarantees a valid normalised probability distribution without any extra constraints.

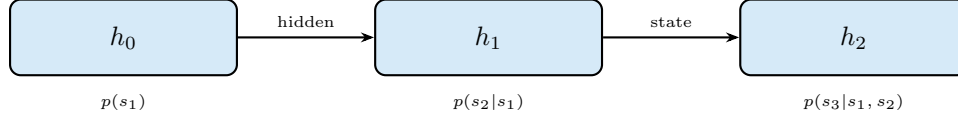
## 4 Is Autoregressive the Same as an RNN?

### Intuition: Short answer

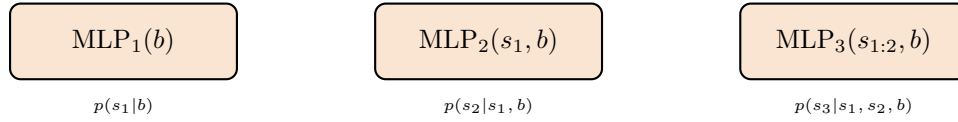
An RNN is *one way* to implement an autoregressive model, but not the only way. The autoregressive property is a *structural constraint on information flow*, not a specific architecture.

### 4.1 Three Ways to be Autoregressive

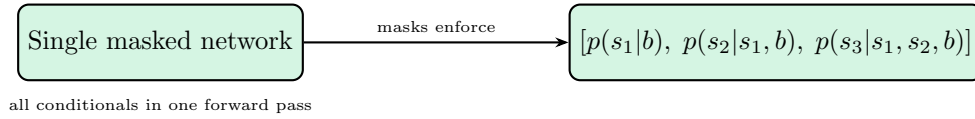
#### RNN



#### Separate MLPs (LowAgent)



#### MADE (HighAgent)



### 4.2 RNN as an Autoregressive Model

An RNN processes sequence left-to-right. At step  $i$ :

$$h_i = \tanh(W_h h_{i-1} + W_x s_{i-1} + W_b b_i + c_h), \quad (11)$$

$$p(s_i | s_{<i}, b) = \sigma(W_o h_i + c_o). \quad (12)$$

The hidden state  $h_i$  compresses all past information into a fixed-size vector. This is exactly the autoregressive property, implemented via *recurrence*.

### Analogy: RNN as a reader

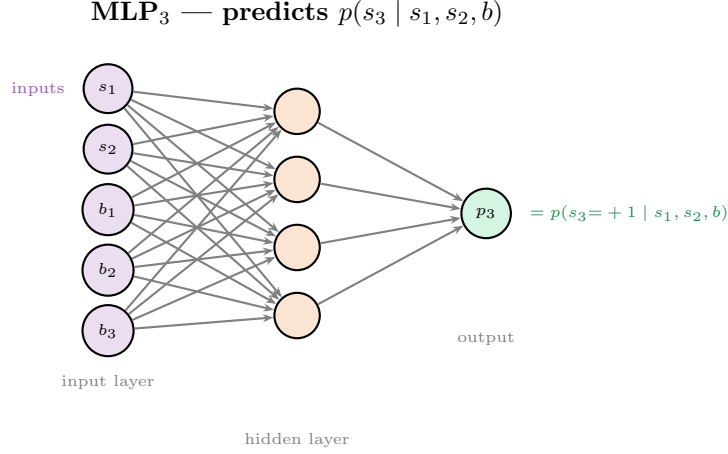
Think of an RNN as a person reading a book. After reading each word, they update their “understanding” (hidden state  $h_i$ ). Their prediction of the next word depends only on their current understanding, not on re-reading the whole book from scratch.

### 4.3 LowAgent: Separate MLPs (Simpler, Faster)

LowAgent avoids recurrence entirely. It uses  $n$  *separate* small MLPs, one per qubit:

$$p_\theta(s_i | s_{<i}, b) = \sigma(\text{MLP}_i([s_1, \dots, s_{i-1}, b_1, \dots, b_n])). \quad (13)$$

The input to  $\text{MLP}_i$  is simply the concatenation of all previous outcomes and the full basis string.



### Intuition: Why not use an RNN here?

For 6–11 qubits, the sequences are very short. The benefit of weight-sharing across positions (the main advantage of RNNs) is minimal. Separate MLPs are simpler to train, easier to parallelize, and avoid vanishing gradient problems. The **LowAgent** trades model capacity for training speed — appropriate when data (shots) are scarce.

## 5 What is a Neural Quantum State (NQS)?

### Analogy: NQS as a compressed photograph

A quantum state on  $n$  qubits lives in a  $2^n$ -dimensional Hilbert space. Storing the full state vector for  $n = 20$  would require  $2^{20} \approx 10^6$  complex numbers. An NQS is like a compressed photograph: instead of storing every pixel, you store the parameters of a neural network that can *regenerate* any pixel on demand. The network is far smaller than the full state but can approximate it well.

Formally, an NQS parameterises the wave function amplitude directly:

$$\psi_\theta(s) = \langle s | \psi \rangle \approx \text{NeuralNet}_\theta(s), \quad (14)$$

or, in the **LowAgent** case, the *conditional Born probability* of measurement outcomes:

$$p_\theta(s \mid b) \approx |\langle s_b | \psi \rangle|^2, \quad (15)$$

where  $|s_b\rangle$  is the basis-rotated computational state. The neural network learns this distribution from data — no explicit wave function is stored.

### 5.1 Why Autoregressive?

For the model to define a valid probability distribution, probabilities must sum to 1:

$$\sum_{s \in \{-1, +1\}^n} p_\theta(s \mid b) = 1. \quad (16)$$

The autoregressive factorisation guarantees this *automatically*: each conditional  $p(s_i \mid s_{<i}, b)$  is a Bernoulli distribution (a single sigmoid), which trivially sums to 1. No normalisation constant needs to be computed, avoiding the intractable partition function that plagues energy-based models.

### Problem

Energy-based model (e.g. RBM)

$$p_\theta(s) = \frac{1}{Z(\theta)} e^{-E_\theta(s)}$$

$$Z(\theta) = \sum_s e^{-E_\theta(s)} \quad \textbf{intractable!}$$

### Solution

Autoregressive model (LowAgent)

$$p_\theta(s) = \prod_i p_\theta(s_i | s_{<i}, b)$$

Each factor is  $\sigma(\cdot) \in (0, 1)$  **always normalised**

## 6 MidAgent: Variational Autoencoder (VAE)

### Analogy: VAE as a sketch artist

Imagine an artist who, upon seeing a measurement outcome  $s$ , first sketches a compressed “essence” of the state (the latent variable  $z$ ), then reconstructs the full picture from that sketch. The VAE does this mathematically: an *encoder* compresses  $(s, b) \rightarrow z$ , a *decoder* reconstructs  $s$  from  $z$  autoregressively.

The VAE introduces a latent variable  $z \in \mathbb{R}^d$  to capture global correlations:

Training objective (ELBO):

$$\mathcal{L}_{\text{VAE}} = \underbrace{\mathbb{E}_{q_\phi(z|s,b)} [\ln p_\theta(s | z, b)]}_{\text{reconstruction}} - \beta \underbrace{D_{\text{KL}}(q_\phi(z | s, b) \| \mathcal{N}(0, I))}_{\text{regularisation}}. \quad (17)$$

The latent  $z$  captures global features of the quantum state that correlate across all qubits — something the independent MLPs of **LowAgent** cannot represent.

## 7 HighAgent: MADE — One Network, All Conditionals

### Analogy: MADE as a parallel factory

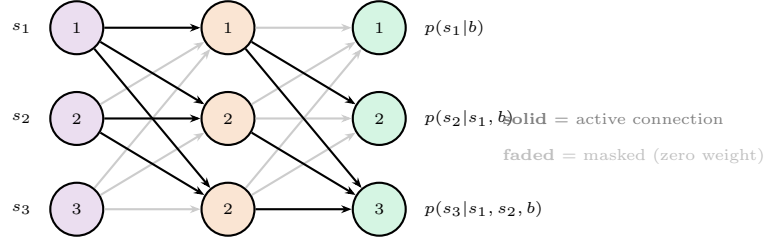
In **LowAgent**, each conditional  $p(s_i | s_{<i}, b)$  is computed by a *separate* MLP — like having  $n$  separate workers each doing one job. MADE uses a *single* network but applies binary masks to the weights, ensuring that the output for qubit  $i$  depends only on inputs  $s_1, \dots, s_{i-1}$ . It is like one worker with a blindfold that blocks them from seeing “future” inputs.

Each weight matrix  $W^{(l)}$  is elementwise-multiplied by a binary mask:

$$\tilde{W}_{ij}^{(l)} = W_{ij}^{(l)} \odot M_{ij}^{(l)}, \quad M_{ij}^{(l)} = \mathbf{1} \left[ m^{(l)}(i) \geq m^{(l-1)}(j) \right], \quad (18)$$

where  $m^{(l)}(i) \in \{1, \dots, n\}$  is the *order* assigned to unit  $i$  in layer  $l$ .

### MADE: masked connections (order labels shown inside units)



The key advantage: **all  $n$  conditionals are computed in a single forward pass**, making HighAgent faster to evaluate than  $n$  separate MLP calls during EFE scoring.

## 8 Comparison of the Three Agents

| Agent     | Architecture      | Like an RNN? | Latent space? | Forward passes        |
|-----------|-------------------|--------------|---------------|-----------------------|
| LowAgent  | $n$ separate MLPs | No           | No            | $n$ (one per qubit)   |
| MidAgent  | AR-VAE            | No           | Yes           | $n$ + encoder         |
| HighAgent | MADE-VAE          | No           | Yes           | 1 (masks do the work) |
| Random    | LowAgent (no EFE) | No           | No            | $n$                   |

Table 1: Architectural comparison of the four benchmark agents.

### Key Idea: None of them are RNNs

All three models implement the autoregressive factorisation *without recurrence*. This avoids vanishing gradients, allows full parallelism during training, and is well-suited to the short sequences ( $n \leq 20$ ) typical of quantum tomography experiments.

## 9 The Active Learning Pipeline

## 10 Relationship to Classical Shadow Tomography

The RandomAgent is classical shadow tomography. The Low/Mid/HighAgent add active inference on top, replacing the uniform random policy with an EFE-driven policy that targets stabilizer-relevant measurements.

| Component             | Classical Shadows                | This Work                              |
|-----------------------|----------------------------------|----------------------------------------|
| Basis selection       | Uniform random $\{X, Y, Z\}^n$   | EFE-adaptive                           |
| State representation  | Empirical average $\hat{\rho}$   | Neural $p_\theta(s b)$                 |
| Channel inversion     | Explicit $\mathcal{M}^{-1}$      | Implicit (likelihood)                  |
| Observable estimation | Unbiased Pauli estimator         | $\mathbb{E}_{p_\theta}[\hat{v}_{K_i}]$ |
| Sample complexity     | $\mathcal{O}(\log M/\epsilon^2)$ | Empirically lower                      |

Table 2: Classical shadow tomography vs. neural active inference.

## 11 Summary

1. **Autoregressive models** decompose  $p(s|b)$  using the chain rule. This is not unique to RNNs — it can be implemented with separate MLPs (LowAgent) or masked networks (HighAgent).
2. **NQS** is the idea of using a neural network to represent a quantum state implicitly, avoiding exponential memory.
3. **VAE** adds a latent space to capture global correlations across qubits, at the cost of a more complex training objective.
4. **MADE** achieves the same autoregressive property as separate MLPs but in a single forward pass, via binary weight masks.
5. **RandomAgent** is classical shadow tomography — the correct statistical baseline.
6. **Active inference** replaces random basis selection with EFE-driven selection, targeting measurements that most reduce uncertainty about the stabilizers.